

UCS503 – Software Engineering Project

Academic Year 2026–2027

Real-Time Flood Risk AI Prediction System

A Data Fusion and Machine Learning Pipeline for Dynamic Disaster Threat Mapping

Submitted by:

1024240005 Amit Bishnoi *ML, API & Frontend Lead*

1024240015 Diksha Garg *System Architecture & Data Engineering*

Computer Science and Engineering Department (CSED)

Submitted to:

Dr. Jeelani Asif

Department of Computer Science and Engineering

Repository: <https://github.com/Flood-Prediction-AI-Team/Flood-Prediction-Using-WeatherAPI-and-historical-thread-mapping>

Documentation: <https://Flood-Prediction-AI-Team.github.io/Flood-Prediction-Using-WeatherAPI-and-historical-thread-mapping/>

Prototype Stage Report

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	1
1.1	Background and Context	1
1.1.1	Problem Statement	1
1.2	Project Objectives	1
1.3	Scope of this Prototype	1
2	Methodology and Design	2
2.1	System Architecture	2
2.1.1	High-Level Design	2
2.2	Machine Learning Strategy and Model Selection	2
2.2.1	Evaluated Baseline Models	2
2.2.2	Why Random Forest (The Chosen Architecture)?	3
2.3	Implementation Details	3
2.3.1	Data Transformation Pipeline in <code>feature_engineering.py</code>	3
2.3.2	Module 3: Frontend Dashboard and Visualization	4

Chapter 1

Introduction

1.1 Background and Context

Traditional flood prediction models often rely solely on static geographical contours or standalone historical archives. While modern meteorological APIs provide high-precision real-time atmospheric metrics, they frequently lack historical baseline disaster contexts. Conversely, traditional hazard maps are drawn once and rarely updated, rendering them ineffective during sudden, severe storms. Live weather data and historical disaster records exist in isolated silos, making it difficult for responders to instantly correlate a storm with historical vulnerability.

Before building our complete platform, we evaluated the feasibility of combining REST weather metrics with spatial centroid proximity queries. The proof-of-concept confirmed that fusing asynchronous OpenWeatherMap payloads with historical DFO and EM-DAT disaster records enables accurate, low-latency risk classification.

1.1.1 Problem Statement

Given a target location, we want a system that ingests live meteorological data via external APIs, executes spatial proximity queries against historical flood centroids within a 100km radius, fuses these data streams into feature vectors, and produces a verified threat classification backed by an interactive geospatial dashboard.

1.2 Project Objectives

1. Build an automated data ingestion pipeline fetching real-time meteorological metrics via the OpenWeatherMap API with spatial proximity indexing using the Haversine formula.
2. Engineer a machine learning pipeline (Random Forest classifier) trained on fused historical data and synthetic overlapping datasets to prevent overfitting.
3. Deploy a responsive frontend dashboard using HTML/CSS, JavaScript, and Leaflet.js to render threat levels and geospatial radius maps in real time.
4. Implement robust mock weather fallback mechanisms to ensure continuous operation during third-party API rate limits.
5. Provide comprehensive documentation hosted via MkDocs and GitHub Pages alongside complete unit testing coverage.

1.3 Scope of this Prototype

Following the course roadmap, the prototype stage delivers the full end-to-end pipeline: historical dataset cleaning and merging, Haversine spatial indexing, OpenWeatherMap API integration with fail-safe fallback logic, Random Forest threat classification, and an interactive Leaflet.js frontend dashboard.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

The system is organized as a decoupled 3-tier architecture mapping onto our division of labor: data engineering, machine learning backend, and frontend UI visualization.

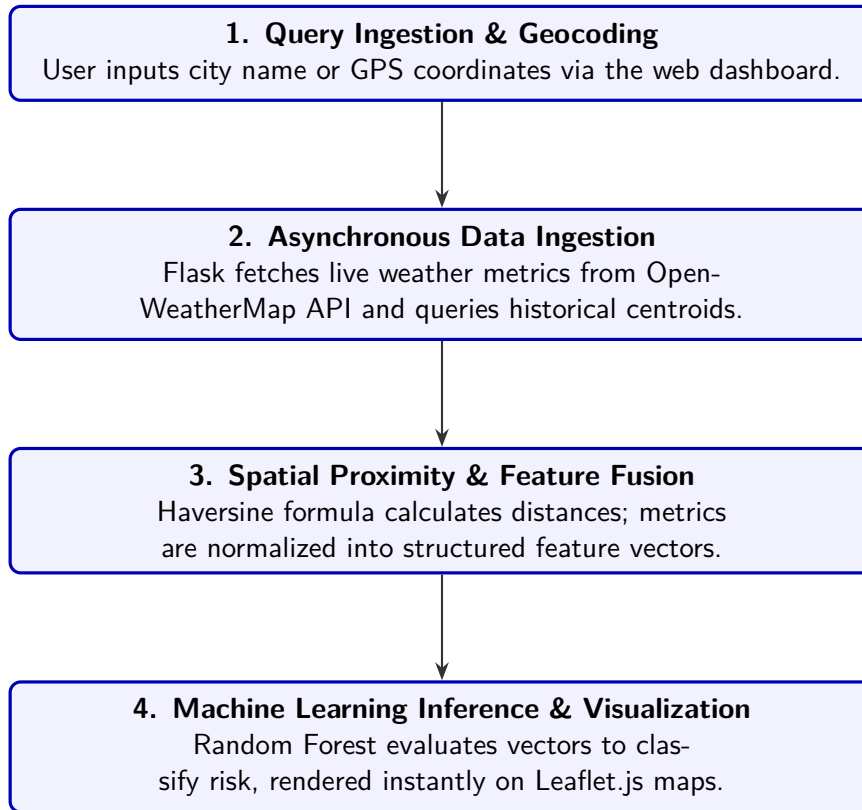


Figure 2.1: High-Level Data Ingestion and Processing Flow.

2.2 Machine Learning Strategy and Model Selection

To reliably evaluate complex, non-linear environmental conditions against historical baselines, selecting the appropriate machine learning architecture is critical. The system relies on a classification paradigm to categorize imminent threat levels.

2.2.1 Evaluated Baseline Models

Prior to finalizing the prediction engine, several algorithmic approaches were considered:

- **Logistic Regression:** Served as the initial linear baseline. While computationally inexpensive, it struggled to capture the complex, non-linear interactions between atmospheric volatility (e.g., rapid humidity spikes) and geographical proximity.
- **Support Vector Machines (SVM):** Effective at drawing decision boundaries in high-dimensional spaces, but computationally heavy and highly sensitive to unscaled environmental outliers during live API fetching.

2.2.2 Why Random Forest (The Chosen Architecture)?

The final pipeline utilizes a **Random Forest Classifier**. It constructs multiple decision trees during training to output a majority-voted class. This ensemble approach was chosen for three distinct advantages:

1. **Noise Robustness:** Averages multiple trees to filter out sudden anomalies and outliers frequently found in live weather API payloads.
2. **Non-Linear Fusion:** Seamlessly maps continuous atmospheric metrics (rain, wind) against discrete spatial counts (past regional floods).
3. **Interpretability:** Extracts feature importance automatically, allowing the system to verify exactly which metrics triggered a high-risk alert.

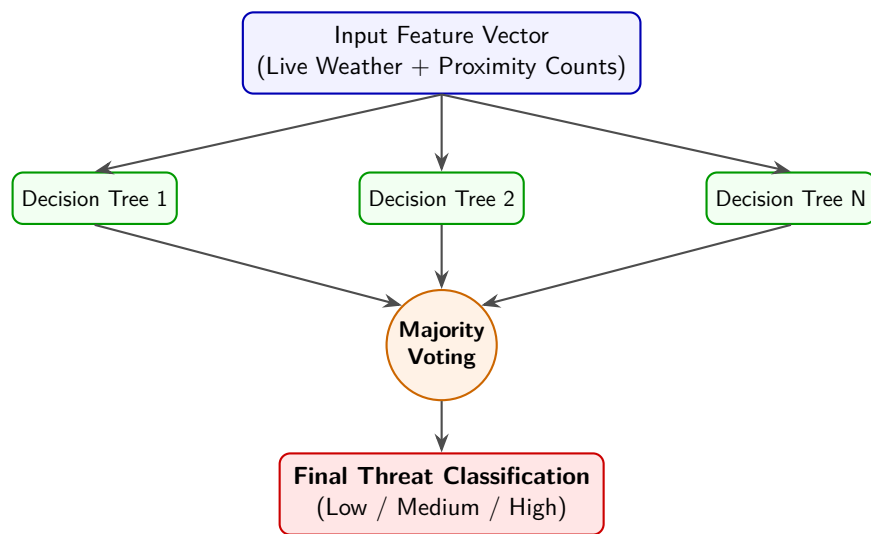


Figure 2.2: Random Forest Ensemble Architecture for Risk Classification.

2.3 Implementation Details

2.3.1 Data Transformation Pipeline in `feature_engineering.py`

The core data transformation and preprocessing logic is executed within the `feature_engineering.py` script. Running this module processes raw heterogeneous data streams and outputs a standardized, synchronized feature array ready for model training and live inference.

- **Where the Data Comes From (Input Sources):** The script ingests two primary data origins:
 1. *Historical Disaster Archives:* Cleaned CSV records containing global flood epicenters, severity indices, and past occurrence counts stored in `Code/Merged_dataset/fused_historical_database.csv`.
 2. *Live Meteorological Payloads:* Real-time atmospheric JSON responses fetched dynamically via the OpenWeatherMap REST API (containing temperature, humidity, wind speed, and precipitation metrics).
- **Data Transformation and Output Generated:** Upon execution, `feature_engineering.py` performs coordinate normalization, calculates geodesic

proximity counts within a 100km radius using the Haversine formula, and constructs standardized multi-variable feature vectors. It outputs normalized feature matrices along with serialized feature scalers and encoders.

- **Where the Data Goes (Destinations):** The processed feature vectors and normalized arrays are passed directly to the Random Forest training pipeline (`train_model.ipynb`) to serialize model weights into `train_model.pkl`. During live execution, the backend Flask server (`backend.py`) invokes these engineered features to evaluate incoming user queries and transmit classified risk levels to the Leaflet frontend dashboard.

2.3.2 Module 3: Frontend Dashboard and Visualization

This module renders the user interface (`Code/Frontend/`), featuring location search bars, dynamic risk indicator cards, and Leaflet.js map rendering that automatically executes "fly-to" animations and visualizes 50km geospatial threat radius circles. Figure 2.3 illustrates the deployed prototype interface.

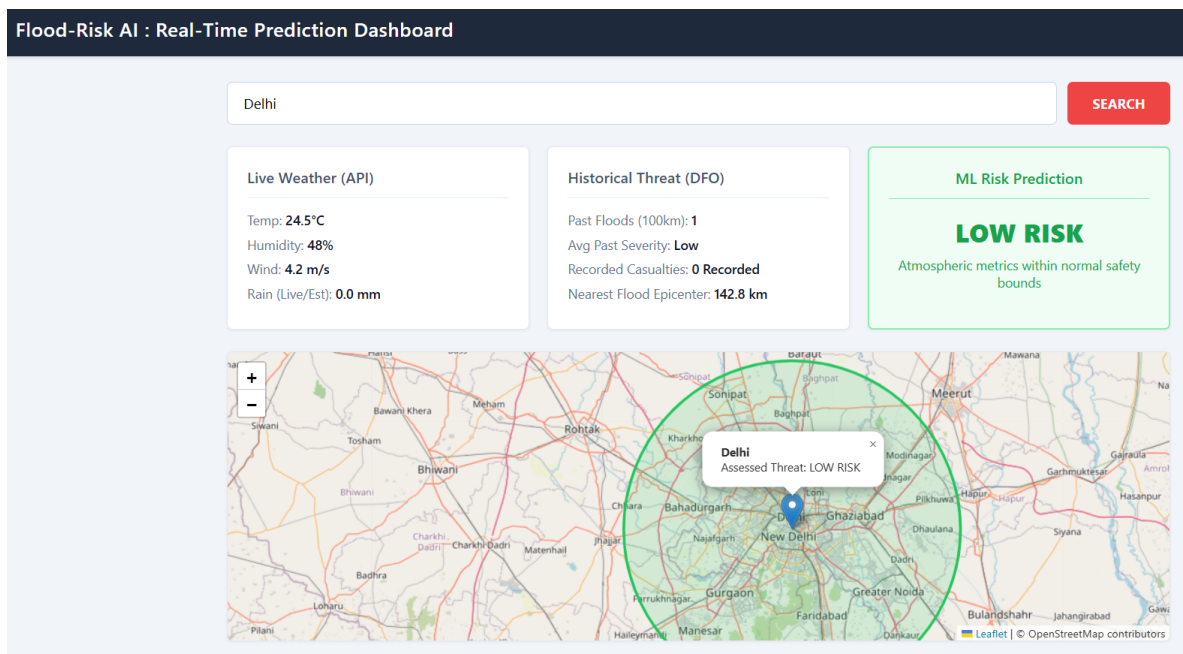


Figure 2.3: Interactive Leaflet.js Frontend Dashboard displaying live weather metrics and assessed threat levels.